

RAPPORT DE PROJET

Phase 1 — Initialisation du projet

Projet : Secure API — Cybersécurité

1. Introduction et contexte du projet

Ce rapport documente la Phase 1 du projet Secure API, un projet de cybersécurité réalisé dans le cadre d'une recherche d'alternance. L'objectif global du projet est de construire une API REST sécurisée en Node.js et TypeScript, démontrant des compétences techniques en développement backend et en cybersécurité.

La Phase 1 correspond à l'initialisation complète de l'environnement de travail : création du dépôt GitHub, configuration du projet Node.js/TypeScript, installation des dépendances et mise en place d'un premier serveur fonctionnel.

1.1 Objectifs de la Phase 1

- Initialiser un projet Node.js avec TypeScript
- Installer toutes les bibliothèques nécessaires au projet
- Mettre en place la structure de dossiers professionnelle
- Lancer un premier serveur Express fonctionnel
- Rédiger un README professionnel compréhensible

2. Définitions des concepts clés

2.1 Outils et technologies

Node.js : Environnement d'exécution JavaScript côté serveur. Contrairement au JavaScript classique qui s'exécute dans le navigateur, Node.js permet d'exécuter du JavaScript sur un serveur. C'est la base de tout projet backend JavaScript.

TypeScript : Surcouche de JavaScript développée par Microsoft qui ajoute un système de types statiques. Cela signifie que le développeur précise exactement le type de chaque variable (nombre, texte, objet...). TypeScript détecte les erreurs avant l'exécution du code, ce qui le rend plus robuste et plus sûr — particulièrement important dans un contexte de cybersécurité.

Express : Framework (cadre de travail) pour Node.js qui simplifie la création de serveurs web et d'APIs REST. Il gère la réception des requêtes HTTP, le routage (vers quelle fonction envoyer chaque requête) et l'envoi des réponses.

npm (Node Package Manager) : Gestionnaire de bibliothèques pour Node.js. C'est l'équivalent d'un App Store pour les développeurs : il permet de télécharger et d'installer du code réutilisable créé par d'autres développeurs.

Git : Système de contrôle de version qui enregistre l'historique de toutes les modifications apportées à un projet. Il permet de revenir à une version précédente, de collaborer à plusieurs, et de suivre qui a modifié quoi et quand.

GitHub : Plateforme en ligne qui héberge des dépôts Git. C'est le réseau social des développeurs : on y publie son code, on peut collaborer avec d'autres, et les recruteurs y consultent les projets des candidats.

API REST (Application Programming Interface Representational State Transfer) : Interface qui permet à différentes applications de communiquer entre elles via le protocole HTTP. Une API REST expose des endpoints (points d'accès) identifiés par des URLs. Chaque endpoint répond à un type de requête précis (GET, POST, PUT, DELETE) et renvoie des données en format JSON.

JSON (JavaScript Object Notation) : Format léger d'échange de données, lisible à la fois par les humains et les machines. C'est le format standard utilisé par les APIs pour envoyer et recevoir des données. Exemple : { "nom": "Jean", "age": 25 }

HTTP (HyperText Transfer Protocol) : Protocole de communication utilisé sur le web pour l'échange de données entre un client (navigateur, application) et un serveur. Chaque communication HTTP comprend une requête (du client vers le serveur) et une réponse (du serveur vers le client).

2.2 Méthodes HTTP

Les requêtes HTTP utilisent différentes méthodes selon l'action souhaitée :

Méthode	Action	Exemple dans notre projet
GET	Récupérer des données	GET /users → lister les utilisateurs
POST	Créer une ressource	POST /register → créer un compte
PUT	Modifier une ressource	PUT /users/1 → modifier un utilisateur
DELETE	Supprimer une ressource	DELETE /users/1 → supprimer un utilisateur

2.3 Codes de statut HTTP

Chaque réponse HTTP est accompagnée d'un code numérique indiquant le résultat de la requête :

Code	Signification	Usage
200 OK	Succès	Requête traitée avec succès
201 Created	Ressource créée	Compte utilisateur créé
400 Bad Request	Mauvaise requête	Données manquantes ou invalides
401 Unauthorized	Non authentifié	Token JWT absent ou invalide
404 Not Found	Introuvable	Route ou ressource inexistante
500 Internal Server Error	Erreur serveur	Erreur inattendue côté serveur

2.4 Fichiers de configuration

package.json : Fichier JSON à la racine du projet qui sert de carte d'identité. Il contient le nom du projet, sa version, la liste de toutes les bibliothèques utilisées (dépendances), et les scripts npm (commandes raccourcies comme npm run dev).

tsconfig.json : Fichier de configuration de TypeScript. Il indique au compilateur TypeScript comment transformer le code TypeScript en JavaScript. On y définit notamment le dossier source (src/), le dossier de sortie (dist/) et le niveau de rigueur des vérifications (strict: true).

.env : Fichier contenant les variables d'environnement — c'est-à-dire les informations sensibles et spécifiques à l'environnement (clés secrètes, mots de passe de bases de données, numéros de ports). Ce fichier ne doit JAMAIS être publié sur GitHub.

.gitignore : Fichier qui liste tous les fichiers et dossiers que Git doit ignorer et ne jamais envoyer sur GitHub. Dans un projet Node.js, on y exclut notamment node_modules/ (trop volumineux) et .env (contient des secrets).

3. Étapes réalisées

3.1 Création du dépôt GitHub

Avant même d'écrire une ligne de code, nous avons créé le dépôt GitHub. Cette approche professionnelle (souvent appelée Git-first) présente plusieurs avantages :

- Le code est sauvegardé en ligne dès le début
- L'historique de progression est visible par les recruteurs
- La collaboration future est facilitée

Le dépôt a été configuré avec les options suivantes :

Option	Choix	Raison
Visibilité	Public	Accessible a tout publique
README	Activé	Page d'accueil automatique du projet
.gitignore	Node	Protection automatique des fichiers sensibles Node.js
Licence	MIT	Bonne pratique open source standard

3.2 Clonage et ouverture dans VS Code

Le dépôt GitHub a été cloné en local avec la commande :

```
git clone https://github.com/Dkls7777/-secure-api-project.git
```

Visual Studio Code (VS Code) a été utilisé comme éditeur de code. C'est l'éditeur le plus utilisé par les développeurs professionnels. Son terminal intégré permet de taper des commandes sans quitter l'éditeur.

3.3 Initialisation du projet Node.js

La commande suivante crée le fichier package.json avec les informations de base du projet :

```
npm init -y
```

L'option -y (yes) répond automatiquement oui à toutes les questions de configuration, créant un package.json avec des valeurs par défaut.

3.4 Installation des dépendances

Deux catégories de dépendances ont été installées :

Dépendances de production (utilisées en production) :

```
npm install express better-sqlite3 bcryptjs jsonwebtoken dotenv
```

Bibliothèque	Rôle
express	Création du serveur web et des routes API
better-sqlite3	Base de données légère stockée dans un fichier
bcryptjs	Chiffrement des mots de passe

jsonwebtoken	Création et vérification des tokens JWT
dotenv	Lecture des variables d'environnement depuis .env

Dépendances de développement (uniquement en développement) :

```
npm install --save-dev typescript ts-node nodemon @types/node @types/express
...
```

Bibliothèque	Rôle
typescript	Compilateur TypeScript
ts-node	Exécution directe de TypeScript sans compilation préalable
nodemon	Redémarrage automatique du serveur lors des modifications
@types/*	Définitions de types TypeScript pour les bibliothèques

3.5 Configuration de TypeScript

La commande `npx tsc --init` génère un fichier `tsconfig.json`. Ce fichier a été personnalisé pour définir :

- `rootDir: ./src` — le code source est dans le dossier `src/`
- `outDir: ./dist` — le code compilé ira dans `dist/`
- `strict: true` — vérification rigoureuse de tous les types
- `esModuleInterop: true` — compatibilité entre les modules CommonJS et ES

3.6 Structure professionnelle des dossiers

Une structure de dossiers claire a été mise en place dans le répertoire `src/` :

Dossier	Rôle
<code>src/routes/</code>	Définition des URLs de l'API et leur méthode HTTP
<code>src/controllers/</code>	Logique métier — traitement des requêtes et construction des réponses
<code>src/middleware/</code>	Fonctions intermédiaires — vérification JWT, validation des données
<code>src/database/</code>	Connexion et initialisation de la base de données

Cette architecture suit le pattern MVC (Model-View-Controller) adapté aux APIs : chaque couche a une responsabilité précise, ce qui facilite la maintenance et la compréhension du code.

3.7 Scripts npm configurés

Les scripts suivants ont été configurés dans package.json :

Commande	Script exécuté	Usage
npm run dev	nodemon --exec ts-node src/index.ts	Développement avec rechargement automatique
npm run build	tsc	Compilation TypeScript → JavaScript pour production
npm start	node dist/index.js	Lancement du serveur en production

3.8 Premier serveur Express

Le fichier src/index.ts constitue le point d'entrée de l'application. Il initialise Express, configure les middlewares (dont express.json()) pour lire le JSON des requêtes, et expose un premier endpoint /health.

Le endpoint /health est une bonne pratique de production : il permet à un outil de monitoring de vérifier rapidement que le serveur fonctionne correctement. Il répond avec :

```
{ "status": "OK", "message": "API opérationnelle", "timestamp": "..." }
```

3.9 Protection des fichiers sensibles

Deux vérifications importantes ont été effectuées pour la sécurité :

- Le fichier .env (contenant les secrets) est bien listé dans .gitignore
- Le fichier database.sqlite (base de données) a été retiré du dépôt GitHub

Ces précautions évitent la fuite accidentelle d'informations sensibles sur internet — une erreur que de nombreux développeurs commettent et qui peut avoir de graves conséquences de sécurité.

4. Historique des commits GitHub

Chaque étape significative a fait l'objet d'un commit GitHub avec un message descriptif suivant la convention Conventional Commits :

Commit	Description
feat: initialisation du projet Node.js + TypeScript	Première version fonctionnelle du serveur
docs: ajout du README professionnel	Documentation du projet
security: suppression de database.sqlite du repo	Retrait de la base de données du dépôt public

La convention feat:/docs:/security: dans les messages de commit permet d'identifier rapidement la nature de chaque modification. C'est une pratique professionnelle standard dans les équipes de développement.

5. Bilan et compétences acquises

5.1 Ce qui a été réalisé

Élément	Statut
Dépôt GitHub public et configuré	Réalisé
Projet Node.js + TypeScript initialisé	Réalisé
Toutes les dépendances installées	Réalisé
Structure professionnelle des dossiers	Réalisé
Serveur Express fonctionnel sur port 3000	Réalisé
Endpoint /health opérationnel	Réalisé
README professionnel rédigé	Réalisé
Fichiers sensibles protégés	Réalisé

5.2 Compétences démontrées

- Maîtrise de l'environnement de développement Node.js/TypeScript
- Utilisation professionnelle de Git et GitHub
- Compréhension de l'architecture d'une API REST
- Application des bonnes pratiques de sécurité (protection des secrets)
- Organisation et structuration d'un projet professionnel

